



## **UNIVERSIDAD DEL VALLE DE GUATEMALA**

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3069 Computación Paralela y Distribuida

Ciclo 2 de 2026

### **Laboratorio 03**

Búsqueda de una clave AES por fuerza bruta:  
versión secuencial y versión paralela con Open MPI

#### **Integrantes:**

María José Girón Isidro

Cindy Gualim Perez

Leonardo Dufrey Mejía Mejía

Guatemala, 2 de octubre de 2026

# Índice

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>1. Investigación sobre AES.....</b>                           | <b>3</b>  |
| 1.1 ¿Qué es AES? .....                                           | 3         |
| 1.2 Campos de aplicación .....                                   | 3         |
| 1.3 Cifrado y descifrado con AES-128 .....                       | 4         |
| 1.3.1 Tamaño de la clave y procesamiento por bloques .....       | 4         |
| 1.3.2 Pasos para cifrar .....                                    | 4         |
| 1.3.3 Pasos para descifrar.....                                  | 5         |
| 1.3.4 Principales transformaciones .....                         | 5         |
| 1.4 Diagrama de flujo .....                                      | 6         |
| <b>2. Análisis del programa secuencial .....</b>                 | <b>7</b>  |
| 2.1 ¿Utiliza correctamente AES-128?.....                         | 7         |
| 2.2 Compilación, ejecución y tiempo .....                        | 7         |
| 2.3 Errores conceptuales y limitaciones.....                     | 8         |
| 2.3.1 Construcción de la clave .....                             | 8         |
| 2.3.2 El modo ECB .....                                          | 8         |
| 2.3.3 Verificación de las claves con el texto original .....     | 9         |
| 2.3.4 Todas las claves posibles frente a las que se prueban..... | 9         |
| 2.3.5 Otras limitaciones .....                                   | 9         |
| 2.4 Mejoras propuestas e implementadas .....                     | 10        |
| <b>3. Versión paralela con Open MPI .....</b>                    | <b>11</b> |
| 3.1 Diseño de la solución .....                                  | 11        |
| 3.2 Verificación de resultados.....                              | 13        |
| 3.3 Tiempos de ejecución y Speedup.....                          | 14        |
| 3.4 Análisis de resultados .....                                 | 15        |
| <b>4. Archivos entregados.....</b>                               | <b>17</b> |
| <b>Referencias.....</b>                                          | <b>17</b> |

# 1. Investigación sobre AES

## 1.1 ¿Qué es AES?

AES (Advanced Encryption Standard, o Estándar de Cifrado Avanzado) es el método de cifrado que el NIST, la agencia de estándares de Estados Unidos, adoptó en 2001 en el documento FIPS 197. Surgió de un concurso público para reemplazar a DES, que ya se podía romper probando todas sus claves. El ganador fue el algoritmo Rijndael, creado por los belgas Joan Daemen y Vincent Rijmen.

Sus características principales son las siguientes:

- Es simétrico, es decir, se usa la misma clave para cifrar y para descifrar.
- Trabaja por bloques: siempre procesa el mensaje en pedazos de 16 bytes (128 bits).
- En cada vuelta, o ronda, sustituye y revuelve todos los bytes del bloque, de modo que un cambio pequeño en la entrada cambia por completo la salida.

AES tiene tres variantes, que se diferencian por el tamaño de la clave y por la cantidad de rondas que aplican, como se resume en la Tabla 1.

**Tabla 1.** *Variantes de AES*

| Variante | Tamaño de la clave  | Rondas |
|----------|---------------------|--------|
| AES-128  | 128 bits (16 bytes) | 10     |
| AES-192  | 192 bits (24 bytes) | 12     |
| AES-256  | 256 bits (32 bytes) | 14     |

Hoy no existe una forma práctica de romper AES cuando se usa bien. El mejor ataque conocido contra AES-128, publicado en 2011, apenas ahorra una parte mínima del trabajo. Con una clave aleatoria, la única opción sigue siendo probar claves una por una, y AES-128 tiene unos 340 sextillones de claves posibles (2 elevado a 128).

## 1.2 Campos de aplicación

AES está presente en casi todo lo que usamos a diario, aunque no lo notemos. En la Tabla 2 se describen cinco campos donde se aplica.

**Tabla 2.** *Campos de aplicación de AES*

| <b>Campo</b>                   | <b>Ejemplos</b>                                                     | <b>Cómo se usa AES</b>                                                                           |
|--------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Navegación web                 | Páginas con HTTPS, conexiones SSH                                   | Protege lo que viaja entre el navegador y el servidor, como contraseñas y datos bancarios.       |
| Redes Wi-Fi                    | WPA2 y WPA3                                                         | Cifra todo lo que se envía por la red inalámbrica para que otra persona cercana no pueda leerlo. |
| Discos y teléfonos             | BitLocker en Windows, FileVault en Mac, cifrado de Android y iPhone | Cifra el disco completo; si roban el equipo, los archivos no se pueden leer sin la clave.        |
| Redes privadas virtuales (VPN) | IPsec, OpenVPN                                                      | Crea un túnel cifrado para trabajar desde casa como si se estuviera en la oficina.               |
| Mensajería                     | Signal, WhatsApp                                                    | Cifra cada mensaje con AES-256 para que solo el destinatario pueda leerlo.                       |

### **1.3 Cifrado y descifrado con AES-128**

En AES-128, la clave de 16 bytes se convierte primero en 11 claves de ronda. Después, cada bloque del mensaje pasa por 10 rondas de transformaciones. Para descifrar se hace el camino de regreso, con las mismas claves de ronda pero en orden contrario.

#### ***1.3.1 Tamaño de la clave y procesamiento por bloques***

La clave de AES-128 mide 128 bits, es decir, 16 bytes. El mensaje también se procesa en bloques de 16 bytes, y cada bloque se acomoda en una tabla de 4 filas por 4 columnas a la que se le llama estado. Si el mensaje es más largo, se parte en varios bloques; si el último bloque queda incompleto, se rellena hasta completar los 16 bytes.

La forma de encadenar los bloques se conoce como modo de operación. En el modo ECB cada bloque se cifra por separado, mientras que en modos como CBC o GCM se usa además un valor inicial aleatorio (IV), para que el mismo mensaje no produzca siempre el mismo resultado.

#### ***1.3.2 Pasos para cifrar***

1. Dividir el mensaje en bloques de 16 bytes y rellenar el último si hace falta.
2. Expandir la clave: a partir de la clave original se generan 11 claves de ronda, de la K0 a la K10.
3. Ronda inicial: combinar el bloque con la clave K0 (AddRoundKey).

4. Rondas 1 a 9: aplicar SubBytes, ShiftRows, MixColumns y AddRoundKey con la clave de esa ronda.
5. Ronda 10: aplicar SubBytes, ShiftRows y AddRoundKey con K10. Esta última ronda no lleva MixColumns.
6. El resultado es el bloque cifrado.

### ***1.3.3 Pasos para descifrar***

1. Expandir la misma clave para obtener las mismas 11 claves de ronda.
2. Combinar el bloque cifrado con la clave K10.
3. Rondas 9 a 1: aplicar los pasos inversos, que son InvShiftRows, InvSubBytes, AddRoundKey con la clave de la ronda e InvMixColumns.
4. Ronda final: aplicar InvShiftRows, InvSubBytes y AddRoundKey con K0.
5. Quitar el relleno y unir los bloques para recuperar el mensaje.

### ***1.3.4 Principales transformaciones***

Cada ronda combina cuatro transformaciones, que se describen en la Tabla 3.

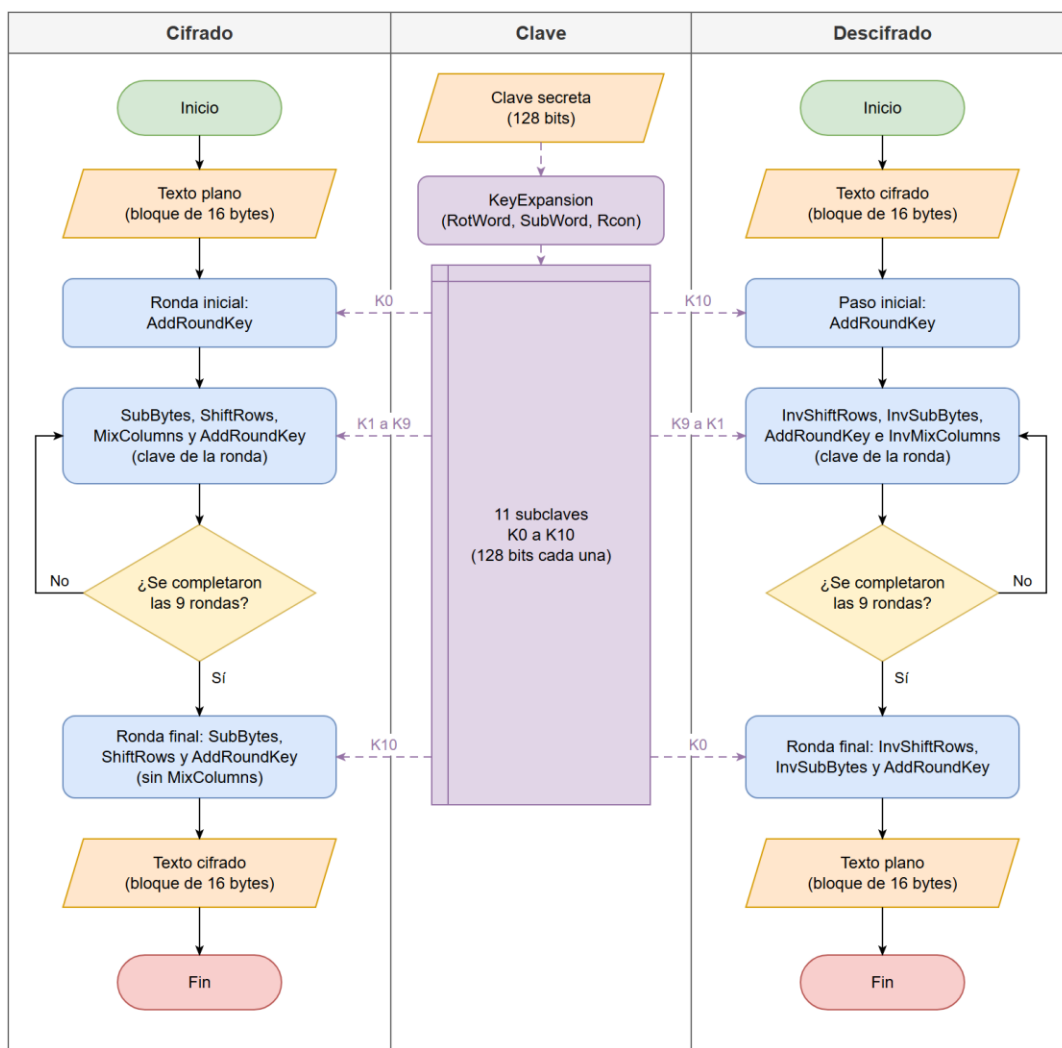
**Tabla 3.** *Transformaciones de AES*

| Transformación | Qué hace                                                                                                                     | Para qué sirve                                                                   | Cómo se deshace                                                  |
|----------------|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------|
| SubBytes       | Cambia cada byte por otro usando una tabla fija llamada S-box.                                                               | Es la parte no lineal del algoritmo; impide resolverlo con ecuaciones sencillas. | Con la tabla inversa (InvSubBytes).                              |
| ShiftRows      | Desplaza las filas del estado: la primera se queda igual, la segunda se corre una posición, la tercera dos y la cuarta tres. | Mezcla los bytes entre columnas.                                                 | Desplazando hacia el otro lado (InvShiftRows).                   |
| MixColumns     | Combina los cuatro bytes de cada columna con una multiplicación especial.                                                    | Hace que cambiar un solo byte afecte a toda la columna.                          | Con la operación inversa (InvMixColumns).                        |
| AddRoundKey    | Combina el estado con la clave de la ronda mediante la operación XOR.                                                        | Es el único paso donde entra la clave.                                           | Repitiendo la misma operación, porque XOR se deshace a sí mismo. |

Gracias a ShiftRows y MixColumns juntas, después de solo dos rondas cada byte del resultado ya depende de los 16 bytes de entrada.

## 1.4 Diagrama de flujo

La Figura 1 muestra las etapas de cifrado y descifrado de AES-128 y cómo interviene la clave en ambos procesos. La clave se expande una sola vez en 11 subclaves, también llamadas claves de ronda. Al cifrar se usan de la K0 a la K10, y al descifrar se usan al revés, de la K10 a la K0, junto con los pasos inversos de cada ronda. El diagrama se elaboró en draw.io y se entrega también como archivo editable.



**Figura 1.** Diagrama de flujo del cifrado y descifrado con AES-128

## 2. Análisis del programa secuencial

### 2.1 ¿Utiliza correctamente AES-128?

Sí, el programa `busqueda_clave_aes_secuencial.c` usa AES-128 de forma correcta: la clave mide 16 bytes, el mensaje ocupa exactamente un bloque de 16 bytes y se cifra y descifra con la misma clave. Lo que tiene problemas no es AES en sí, sino la manera en que el programa lo usa, como explicamos en la sección 2.3. La Tabla 4 contrasta el programa con lo investigado.

**Tabla 4.** Comparación entre lo investigado y el programa secuencial

| Aspecto                   | Lo que investigamos                                                | Lo que hace el programa                                                          | ¿Es correcto?                                                                                                 |
|---------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Tamaño de la clave        | 128 bits (16 bytes).                                               | Usa una clave de 16 bytes con AES-128 de OpenSSL.                                | Sí en la forma, pero 8 de los 16 bytes siempre valen cero y en la práctica solo se prueban unas pocas claves. |
| Procesamiento por bloques | Bloques de 16 bytes, con relleno si el último queda incompleto.    | El mensaje "Puedes lograrlo!" mide justo 16 bytes y el relleno está desactivado. | Sí, cifra exactamente un bloque, aunque no acepta mensajes de otro tamaño.                                    |
| Rondas y transformaciones | 10 rondas con 11 claves de ronda.                                  | OpenSSL las hace por dentro, aprovechando las instrucciones AES del procesador.  | Sí.                                                                                                           |
| Cifrado y descifrado      | La misma clave sirve para ambos procesos.                          | La misma función cifra con la clave secreta y descifra con cada clave candidata. | Sí.                                                                                                           |
| Modo de operación         | Se recomienda un modo con valor inicial aleatorio, como CBC o GCM. | Usa ECB, sin valor inicial.                                                      | Funciona para un solo bloque, pero no es un modo recomendado.                                                 |

### 2.2 Compilación, ejecución y tiempo

El programa original ya medía su propio tiempo, así que solo tuvimos que instalar OpenSSL, compilarlo y ejecutarlo con los comandos del enunciado:

```
sudo apt update
sudo apt install libssl-dev
gcc -std=c11 -O2 -Wall -Wextra busqueda_clave_aes_secuencial.c \
    -o busqueda_clave_aes_secuencial -lcrypto
./busqueda_clave_aes_secuencial
```

El programa compiló sin advertencias y encontró la clave 12345 en unos 11 milisegundos, como se ve en la Figura 2.



En este ejercicio funciona porque el mensaje tiene un solo bloque, pero obliga a que mida exactamente 16 bytes.

### ***2.3.3 Verificación de las claves con el texto original***

Para saber si una clave es la correcta, el programa compara el resultado con el mensaje original. Es decir, el atacante ya conoce el mensaje que supuestamente quiere descubrir. Este tipo de ataque, en el que se conoce el texto, sí se estudia en criptografía, pero el programa no separa a quien cifra de quien ataca: los dos usan la misma variable.

En un caso real el atacante solo sabría algo parcial, por ejemplo que el mensaje contiene cierta palabra, y eso puede dar falsos positivos. Una mejor opción es que el cifrado incluya un sello de autenticación que confirme cuándo la clave es la correcta.

### ***2.3.4 Todas las claves posibles frente a las que se prueban***

AES-128 tiene 2 elevado a 128 claves posibles, unos 340 sextillones. El programa solo prueba 2 elevado a 20, es decir, 1,048,576 claves; comparado con el total, es prácticamente nada. Al ritmo del programa original, unos 2 millones de claves por segundo, probar todas las claves de AES-128 tomaría unos 5 cuatrillones de años, cientos de billones de veces la edad del universo. Además, si la clave secreta fuera mayor que 1,048,575, el programa terminaría sin encontrarla.

### ***2.3.5 Otras limitaciones***

- En cada intento el programa vuelve a preparar el algoritmo desde cero. Si se prepara una sola vez y luego solo se cambia la clave, la revisión completa baja de 0.50 a 0.15 segundos, más de 3 veces más rápido.
- La clave, el mensaje y la cantidad de claves a probar están fijos en el código; para cambiarlos hay que modificarlo y compilar de nuevo.
- Con la clave 12345 solo se prueba poco más del 1 % de las claves, así que ese tiempo no refleja lo que de verdad cuesta la búsqueda.
- Si ocurre un error, el programa termina sin liberar la memoria que usó, y la clave queda guardada en memoria después de usarse.

## 2.4 Mejoras propuestas e implementadas

Con base en lo anterior hicimos una versión mejorada del programa con los seis cambios de la Tabla 5. Aunque el nuevo modo de cifrado hace más trabajo por cada clave, esta versión revisa todas las claves en unos 0.33 segundos, contra 0.50 del original. El código está en el archivo `busqueda_clave_aes_secuencial_mejorado.c`.

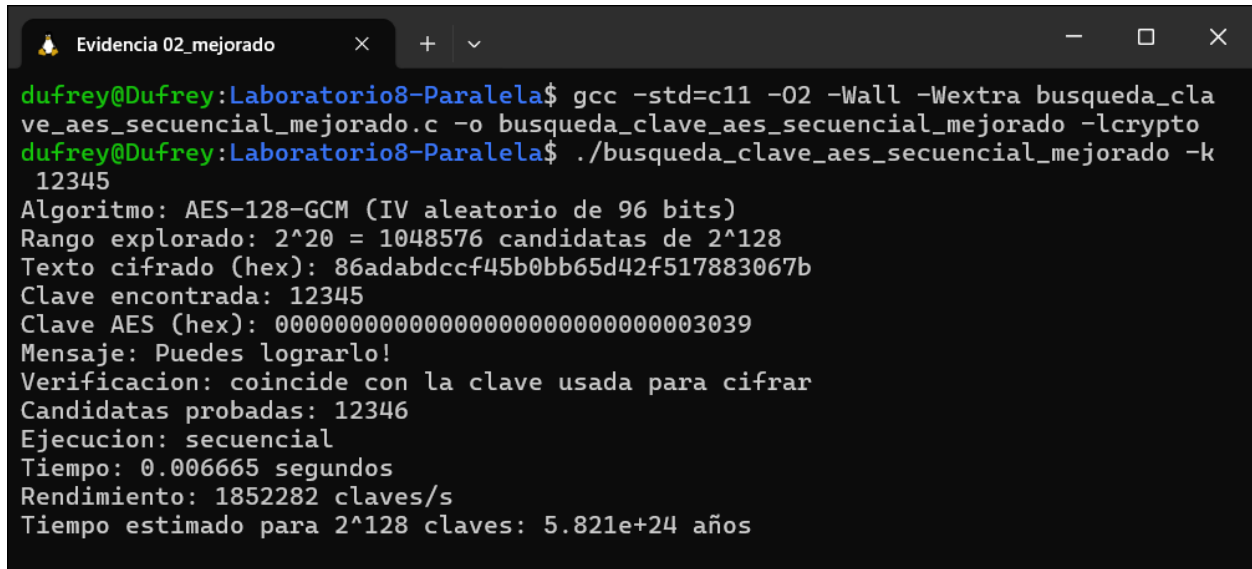
**Tabla 5.** *Mejoras implementadas en la versión secuencial*

| Número | Mejora                                                                                                  | Tipo         | Por qué la hicimos                                                                                                                                                                                          | Propuesta por               |
|--------|---------------------------------------------------------------------------------------------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| 1      | Cambiar el modo ECB por GCM, con un valor inicial aleatorio.                                            | Seguridad    | El mismo mensaje ya no produce siempre el mismo resultado, se aceptan mensajes de distintos tamaños y se agrega un sello que detecta cualquier alteración. Es el modo que usan hoy las páginas web seguras. | María José Girón Isidro     |
| 2      | Verificar las claves con el sello de GCM y no con el mensaje original.                                  | Seguridad    | El buscador solo recibe lo que vería un atacante real: el mensaje cifrado, el valor inicial y el sello. Una clave es correcta cuando el sello coincide.                                                     | María José Girón Isidro     |
| 3      | Elegir la clave secreta al azar y borrarla de la memoria después de usarla.                             | Seguridad    | La clave ya no queda escrita en el programa. Para pruebas repetibles se puede indicar una clave fija.                                                                                                       | Cindy Gualim Perez          |
| 4      | Permitir elegir la cantidad de claves, la clave y el mensaje al ejecutar el programa.                   | Flexibilidad | Ya no hay que modificar el código para probar otros casos, y el programa revisa que los valores sean válidos.                                                                                               | Cindy Gualim Perez          |
| 5      | Preparar el algoritmo una sola vez y en cada intento solo cambiar la clave.                             | Rendimiento  | Evita repetir el mismo trabajo en cada una de las claves.                                                                                                                                                   | Leonardo Dufrey Mejía Mejía |
| 6      | Mostrar cuántas claves se probaron, cuántas por segundo y cuánto tardaría revisar todas las de AES-128. | Rendimiento  | Hace visible el costo real de la búsqueda y lo lejos que está de romper AES-128.                                                                                                                            | Leonardo Dufrey Mejía Mejía |

La forma de construir las claves se dejó igual a propósito. Que haya pocas claves posibles es justamente lo que permite hacer la fuerza bruta en segundos y comparar ambas versiones. La diferencia es que ahora la cantidad de claves se elige al ejecutar el programa.

El programa acepta tres opciones: -b para la cantidad de claves (por ejemplo, -b 20 significa 2 elevado a 20 claves), -k para la clave secreta y -m para el mensaje. Se compila y ejecuta así:

```
gcc -std=c11 -O2 -Wall -Wextra busqueda_clave_aes_secuencial_mejorado.c \
-o busqueda_clave_aes_secuencial_mejorado -lcrypto
./busqueda_clave_aes_secuencial_mejorado -k 12345
```



```
dufrey@Dufrey:Laboratorio8-Paralela$ gcc -std=c11 -O2 -Wall -Wextra busqueda_cla
ve_aes_secuencial_mejorado.c -o busqueda_clave_aes_secuencial_mejorado -lcrypto
dufrey@Dufrey:Laboratorio8-Paralela$ ./busqueda_clave_aes_secuencial_mejorado -k
12345
Algoritmo: AES-128-GCM (IV aleatorio de 96 bits)
Rango explorado: 2^20 = 1048576 candidatas de 2^128
Texto cifrado (hex): 86adabdccf45b0bb65d42f517883067b
Clave encontrada: 12345
Clave AES (hex): 0000000000000000000000000000003039
Mensaje: Puedes lograrlo!
Verificacion: coincide con la clave usada para cifrar
Candidatas probadas: 12346
Ejecucion: secuencial
Tiempo: 0.006665 segundos
Rendimiento: 1852282 claves/s
Tiempo estimado para 2^128 claves: 5.821e+24 años
```

**Figura 3.** *Compilación y ejecución de la versión secuencial mejorada*

Algo que se nota enseguida es que el mensaje cifrado cambia cada vez que se ejecuta el programa, aunque la clave y el mensaje sean los mismos. Eso pasa porque el valor inicial es nuevo en cada ejecución; con ECB siempre saldría igual.

### 3. Versión paralela con Open MPI

#### 3.1 Diseño de la solución

La versión paralela, `busqueda_clave_aes_mpi.c`, parte de la versión mejorada y reparte las claves entre los procesos como quien reparte cartas. Con 4 procesos, el proceso 0 prueba las claves 0, 4, 8 y así sucesivamente; el proceso 1 prueba la 1, 5, 9, y así cada uno. De esta forma cada clave le toca a un solo proceso: ninguna se queda sin probar y ninguna se prueba dos veces.

El programa funciona en estos pasos:

1. El proceso 0 cifra el mensaje y les envía a los demás solo lo que vería un atacante: el mensaje cifrado, el valor inicial y el sello. Los demás procesos nunca conocen la clave ni el mensaje original.

2. Todos los procesos esperan a estar listos y en ese momento empieza a correr el cronómetro.
3. Cada proceso prueba sus claves en grupos de 512.
4. Al terminar cada grupo, los procesos se comunican para saber si alguno ya encontró la clave. Si alguno la encontró, todos se detienen.
5. Si nadie encuentra la clave, todos terminan juntos cuando se acaban las claves, sin quedarse esperando.
6. El proceso 0 junta el tiempo y el total de claves probadas, descifra el mensaje con la clave encontrada y lo muestra.

Elegimos repartir las claves de forma intercalada, y no en bloques seguidos, por un motivo práctico. Si se repartieran en bloques, la clave 12345 le tocaría al proceso 0, que la encontraría al mismo tiempo que la versión secuencial, y agregar procesos no ayudaría en nada. Con el reparto intercalado, cada proceso llega a la clave después de probar solo una parte de las claves.

Para confirmar que ninguna clave se repite ni se omite, ejecutamos el programa con una clave imposible de encontrar: con 2, 3 y 4 procesos se probaron exactamente 1,048,576 claves. También simulamos el reparto con distintas cantidades de procesos y de claves, y cada clave se visitó una sola vez. Además probamos que los procesos se consultaran cada 128, 512, 4,096 y 32,768 claves, sin diferencias claras en el tiempo; nos quedamos con 512 porque permite detenerse rápido.

Para compilar y ejecutar la versión paralela se usan estos comandos:

```
sudo apt install openmpi-bin libopenmpi-dev
mpicc -std=c11 -O2 -Wall -Wextra busqueda_clave_aes_mpi.c \
-o busqueda_clave_aes_mpi -lcrypto
mpirun -np 4 ./busqueda_clave_aes_mpi -k 12345
```



```

Evidencia 04_verificacion x + v
dufrey@Dufrey:Laboratorio8-Paralela$ ./medir_speedup.sh 20 12345 10
Escenario: 2^20 candidatas, clave secreta 12345, 10 repeticiones

Verificacion de resultados:
  Secuencial: 12345 Mensaje: Puedes lograrlo!
  MPI (4):    12345 Mensaje: Puedes lograrlo!
  Ambas versiones recuperan la misma clave y mensaje.

Repeticion 10/10
| Procesos | Tiempo prom (s) | Speedup | Eficiencia |
|-----|-----|-----|-----|
| 1 (sec.) | 0.005502 | 1.00 | 100.0% |
| 2 | 0.003230 | 1.70 | 85.2% |
| 3 | 0.003047 | 1.81 | 60.2% |
| 4 | 0.002507 | 2.19 | 54.9% |

```

**Figura 5.** Verificación de resultados y tiempos con la clave 12345

### 3.3 Tiempos de ejecución y Speedup

Al revisar todas las claves, la versión paralela fue 1.74 veces más rápida con 2 procesos, 1.79 veces con 3 y 2.24 veces con 4. Siempre comparamos contra la versión secuencial mejorada, que es de la que parte la versión paralela.

El Speedup se calcula dividiendo el tiempo de la versión secuencial entre el tiempo de la versión paralela. La eficiencia es el Speedup dividido entre la cantidad de procesos, e indica qué tan bien se aprovecha cada uno. Los resultados al revisar las 1,048,576 claves completas están en la Tabla 7 y en la Figura 6.

**Tabla 7.** Tiempo promedio y Speedup al revisar las 1,048,576 claves

| Cantidad de procesos (n) | Tiempo promedio (segundos) | Speedup calculado | Eficiencia |
|--------------------------|----------------------------|-------------------|------------|
| 1 (secuencial mejorada)  | 0.332120                   | 1.00              | 100.0 %    |
| 2                        | 0.191119                   | 1.74              | 86.9 %     |
| 3                        | 0.185880                   | 1.79              | 59.6 %     |
| 4                        | 0.148097                   | 2.24              | 56.1 %     |

Cada tiempo es el promedio de 10 ejecuciones. En cada repetición corrimos las cuatro configuraciones una tras otra, para que el calentamiento de la laptop afectara a todas por igual. Solo medimos el tiempo de la búsqueda, sin contar lo que tarda en arrancar mpirun.

```

Evidencia 05_speedup x + v - □ ×
dufrey@Dufrey:Laboratorio8-Paralela$ ./medir_speedup.sh 20 1048575 10
Escenario: 2^20 candidatas, clave secreta 1048575, 10 repeticiones

Verificacion de resultados:
  Secuencial: 1048575 Mensaje: Puedes lograrlo!
  MPI (4):    1048575 Mensaje: Puedes lograrlo!
  Ambas versiones recuperan la misma clave y mensaje.

Repeticion 10/10
| Procesos | Tiempo prom (s) | Speedup | Eficiencia |
|-----|-----|-----|-----|
| 1 (sec.) | 0.332120 | 1.00 | 100.0% |
| 2 | 0.191119 | 1.74 | 86.9% |
| 3 | 0.185880 | 1.79 | 59.6% |
| 4 | 0.148097 | 2.24 | 56.1% |

```

**Figura 6.** *Tiempos y Speedup al revisar las 1,048,576 claves completas*

También medimos otros dos casos con el mismo método (Tabla 8). El caso de la clave 12345 se promedió sobre 10 ejecuciones y es el que aparece en la Figura 5; el de 16,777,216 claves se promedió sobre 5 ejecuciones.

**Tabla 8.** *Tiempo promedio y Speedup en otros dos casos*

| Cantidad de procesos (n) | Clave 12345: tiempo (segundos) | Speedup | 16,777,216 claves: tiempo (segundos) | Speedup |
|--------------------------|--------------------------------|---------|--------------------------------------|---------|
| 1 (secuencial mejorada)  | 0.005502                       | 1.00    | 6.068116                             | 1.00    |
| 2                        | 0.003230                       | 1.70    | 3.366495                             | 1.80    |
| 3                        | 0.003047                       | 1.81    | 3.696676                             | 1.64    |
| 4                        | 0.002507                       | 2.19    | 3.382694                             | 1.79    |

Los comandos usados para obtener las tres mediciones fueron:

```

./medir_speedup.sh 20 1048575 10
./medir_speedup.sh 20 12345 10
./medir_speedup.sh 24 16777215 5

```

### 3.4 Análisis de resultados

La versión paralela no llega a ser 4 veces más rápida con 4 procesos, y la razón principal es la computadora, no la comunicación entre procesos. Para comprobarlo ejecutamos varias copias del programa secuencial al mismo tiempo, sin MPI y sin que se comunicaran entre sí. Los resultados están en la Tabla 9.

**Tabla 9.** *Rendimiento de varias copias independientes del programa secuencial*

| Copias al mismo tiempo | Claves por segundo de cada copia | Claves por segundo en total | Mejora máxima posible |
|------------------------|----------------------------------|-----------------------------|-----------------------|
| 1                      | 3,136,387                        | 3,136,387                   | 1.00                  |
| 2                      | 2,695,752                        | 5,391,503                   | 1.72                  |
| 3                      | 2,200,598                        | 6,601,793                   | 2.10                  |
| 4                      | 2,201,006                        | 8,804,024                   | 2.81                  |

Aun sin comunicación, cada copia se vuelve más lenta cuantas más corren a la vez. Con 4 copias la laptop solo alcanza a ser 2.8 veces más rápida, así que el 2.24 de la versión paralela está cerca de ese límite. Las causas que encontramos son las siguientes:

- El procesador es híbrido. El i7-1255U solo tiene 2 núcleos rápidos; los otros 8 son núcleos de eficiencia, más lentos. Con 3 o 4 procesos, algunos terminan en núcleos lentos o compartiendo un núcleo rápido con otro proceso.
- Es un procesador de laptop de 15 W. Cuando trabajan varios núcleos a la vez, cada uno baja su velocidad para no pasarse de ese consumo; por eso cada copia bajó de 3.1 a 2.2 millones de claves por segundo.
- El proceso más lento marca el ritmo. Cada proceso recibe la misma cantidad de claves, así que todos terminan cuando termina el más lento. Si el tercer proceso cae en un núcleo lento, se pierde buena parte de la ganancia, y por eso 3 procesos apenas mejoraron a 2.
- Las pruebas largas rinden menos. Con 16,777,216 claves la búsqueda dura varios segundos, la laptop se calienta, baja su velocidad y el Speedup con 4 procesos cae a 1.79. Además, al usar WSL, Windows decide en qué núcleo corre cada proceso, y eso agrega variación.
- La clave 12345 se encuentra demasiado rápido. La búsqueda dura unos 3 milisegundos, y en un tiempo tan corto pesan más los costos fijos, como la coordinación entre procesos y los núcleos que todavía están arrancando. Por eso el Speedup se queda entre 1.7 y 2.2, lejos del ideal, aunque se agreguen procesos.
- La comunicación casi no cuesta. Los procesos solo se envían unos 300 bytes al inicio y se consultan una vez cada 512 claves. Casi todo el trabajo se puede repartir, así que en una

computadora con núcleos iguales, o en un clúster, el Speedup debería acercarse a la cantidad de procesos.

Por último, aunque usemos 4 procesos y probemos unos 7 millones de claves por segundo, revisar todas las claves de AES-128 tomaría más de un cuatrillón de años. Cada bit que se agrega a la clave duplica el trabajo, mientras que duplicar los procesos, en el mejor de los casos, solo reduce el tiempo a la mitad. Por eso la fuerza bruta no es una amenaza real para AES-128.

## 4. Archivos entregados

**Tabla 10.** Archivos que acompañan este informe

| Archivo                                  | Contenido                                                                  |
|------------------------------------------|----------------------------------------------------------------------------|
| busqueda_clave_aes_secuencial.c          | Programa original, sin cambios.                                            |
| busqueda_clave_aes_secuencial_mejorado.c | Versión secuencial con las seis mejoras.                                   |
| busqueda_clave_aes_mpi.c                 | Versión paralela con Open MPI.                                             |
| Makefile                                 | Compila los tres programas con el comando make.                            |
| medir_speedup.sh                         | Compara los resultados de ambas versiones y mide los tiempos y el Speedup. |
| diagrama/diagrama_aes128.drawio          | Diagrama de flujo de AES-128, editable en draw.io.                         |

Para compilar y probar todo basta con estos comandos:

```
make
./busqueda_clave_aes_secuencial
./busqueda_clave_aes_secuencial_mejorado -k 12345
mpirun -np 4 ./busqueda_clave_aes_mpi -k 12345
./medir_speedup.sh 20 1048575 10
```

## Referencias

National Institute of Standards and Technology. (2001). Advanced Encryption Standard (AES) (FIPS PUB 197). <https://csrc.nist.gov/pubs/fips/197/final>

Open MPI Project. (s. f.). MPI\_Allreduce. [https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI\\_Allreduce.3.html](https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI_Allreduce.3.html)

OpenSSL Project. (s. f.). EVP\_EncryptInit (OpenSSL 3.0). [https://docs.openssl.org/3.0/man3/EVP\\_EncryptInit/](https://docs.openssl.org/3.0/man3/EVP_EncryptInit/)